Small Language Model Jailbreak Defender Framework

*Note: Sub-titles are not captured in Xplore and should not be used

1st Eduardo Castro-Puello
College of Computing, AI, and Cybersecurity
Nova Southeastern University
Fort Lauderdale, FL, USA
ec1784@mynsu.nova.edu

2nd Wei Li
College of Computing, AI, and Cybersecurity
Nova Southeastern University
Fort Lauderdale, FL, USA
lwei@nova.edu

***Abstract*—Large Language Models (LLMs) enable powerful language and task automation but create scalable avenues for abuse, including jailbreaks and fine tuning–driven safety collapse. Small Language Models (SLMs) are increasingly favored for cost, latency, and deployment flexibility, yet exhibit weaker safety priors and heightened vulnerability to adversarial adaptation. Existing defenses are piecemeal and often degrade task utility or efficiency. This paper proposes a modular security alignment framework that enables fine tuned SLMs to approach LLM level robustness while preserving domain performance and low latency. The framework integrates multi model dataset auditing, adversarial artifact conditioning (e.g., toxic token extraction), and an SLM based guardrail wrapper that performs multi step chain of thought intent and output judging. We outline an empirical evaluation agenda focused on reducing attack success rates and unsafe compliance while maintaining high precision and recall on benign tasks and sub second response times, enabling safer SLM deployment in security sensitive domains.**

***Index Terms*—Small Language Model, Fine Tuning, Jailbreak, Security**

I. Introduction

Recent advances in Large Language Models (LLMs) have unlocked impressive capabilities in natural language understanding, multimodal reasoning, and task automation. These models are now embedded in productivity tools, developer workflows, and decision support systems across finance, healthcare, and other regulated sectors. At the same time, their generative power creates new, scalable abuse channels, including targeted social engineering, automated extortion, disinformation generation, malware assistance, and policy violating technical guidance [27]. As adoption expands, adversaries are increasingly incentivized to circumvent safety alignment. A particularly damaging class of attacks leverages fine tuning or lightweight adaptation mechanisms such as LoRA, adapters, or system prompt conditioning to erode refusal behavior and re enable harmful outputs [20] [21]. Empirical work shows that even a few hundred adversarial fine tuning examples can substantially dismantle RLHF aligned safeguards [40]. Concurrently, the ecosystem is shifting toward Small Language Models (SLMs), typically with 1–7B parameters, due to their lower deployment cost, reduced memory footprint, and suitability for edge, on premise, and domain specialized applications [18]. However, SLMs generally possess shallower safety priors, weaker instruction adherence, and are seldom aligned using the extensive human feedback afforded to frontier LLMs. Emerging evidence, including preliminary results from this work, indicates that SLMs are disproportionately vulnerable to jailbreaks, prompt injection, and fine tuning–induced safety collapse. Naive defensive measures such as static rule based filters or heavy safety fine tuning tend to overcorrect, increasing false positives, inducing over refusal of benign content, and degrading core task performance [47]. Current literature offers isolated ingredients for improving safety dataset auditing, red teaming, adversarial prompt generation, and policy filtering but lacks an integrated, resource aware framework that yields LLM grade safety behavior in resource bounded models. This gap is particularly acute in high stakes environments where SLMs are attractive for latency and cost reasons yet operate under strict regulatory and risk constraints. Without a principled approach to simultaneously resist adversarial adaptation, preserve domain performance, and maintain efficiency, organizations face a stark trade off between safety and practicality. This dissertation addresses this gap by designing, implementing, and evaluating a modular security alignment framework for fine tuned SLMs. The proposed framework combines three key components: (a) multi LLM dataset auditing to surface and mitigate unsafe or adversarial training artifacts; (b) adversarial artifact mining, including conditioning on toxic token sequences derived from automated jailbreak systems; and (c) a lightweight SLM based guardrail wrapper that performs multi step chain of thought intent and output judging around a task specialized SLM. The central research question is, whether such a modular, SLM based guardrail framework can enable a $< 7B$ parameter model to achieve safety robustness comparable to larger LLMs, resisting automated jailbreak and fine tuning attacks while preserving domain task performance and SLM class latency. The remainder of the paper demonstrates (a) motivations of the threat model and design requirements for securing

Identify applicable funding agency here. If none, delete this.

SLMs, (b) details of the architecture and training pipeline of the proposed framework, and (c) an empirical evaluation protocol centered on attack success rates, unsafe compliance, safety–utility trade offs, and end to end latency. Together, these contributions aim to advance alignment science for resource bounded models and support safer SLM adoption in regulated, safety critical applications.

II. Related Works

Existing work has established both the breadth of LLM enabled abuse and the particular fragility of alignment under post hoc adaptation. Mylrea [27] documents how large models can be weaponized for scalable social engineering, extortion, disinformation, and technical abuse, motivating strong safety guarantees in deployed systems. A growing body of work shows that fine tuning and related customization channels can rapidly dismantle refusal behavior: even small adversarial or seemingly benign datasets significantly erode RLHF based safeguards in both open and closed models [20] [21] [31] [40] [44]. At the same time, industry is converging on Small Language Models (SLMs) for cost, latency, and deployment flexibility [18], yet empirical studies including those highlighting over refusal from heavy safety tuning and rule based filtering suggest that current defenses induce substantial utility loss and false positives, especially in smaller models [45] [30]. Mekala [26] argues that the field lacks a principled, empirically grounded and resource aware framework that jointly considers robustness, task performance, and efficiency, a gap this dissertation targets for fine tuned SLMs. On the attack side, Yi [41] provides a comprehensive taxonomy of jailbreaks that distinguishes white box from black box settings and categorizes gradient based, logits based, and fine tuning based attacks, offering a unifying lens for evaluating transferability and defense priorities. Within gradient based and discrete optimization methods, universal adversarial suffixes, discrete search, and continuous relaxations have been shown to reliably flip refusals to harmful compliance across diverse models while remaining robust to naïve filters [52] [19] [51] [36] [12]. Adaptive search procedures ranging from random/score based and PGD inspired approaches to query only and proxy guided black box optimization achieve high jailbreak success on both open and commercial APIs, emphasizing model specific adaptivity and scalable automated red teaming [1] [16] [33]. Other work demonstrates that maliciously chosen in context demonstrations can systematically bias downstream predictions [37] and that universal prefixes can propagate harmful compliance through guard rail pipelines, exposing architectural weaknesses in mediated deployments [23]. Logits based attacks exploit access to probability distributions directly or via decoding controls to steer models from refusal toward compliance, showing that log prob exposure and sampling hyperparameters can substantially collapse alignment; these include interrogation based attacks, energy based decoding, and methods that weaponize inherent affirmative biases or explicitly suppress refusal templates [47] [13] [11] [17] [48] [50]. Complementing these prompt and logit level methods, fine tuning based jailbreaks demonstrate that tens to hundreds of adversarial examples, often collected from weaker models, can subvert safety in both open source and frontier systems at low cost and without degrading general utility, highlighting the central challenge of alignment persistence under downstream adaptation [31] [40]; [20] [44].

Defensive research has started to map the design space but remains fragmented, particularly for SLMs. Yi [41] classify defenses into prompt level and model level approaches, outlining strategies such as prompt detection, perturbation, and system prompt hardening, alongside supervised alignment, RLHF, gradient/logit regularization, refinement via adversarial training, and proxy defenses. Empirically, injecting a modest fraction of safety critical data during supervised fine tuning can significantly boost refusal reliability, though at risk of over refusal and utility loss [3], while hybrid manual–automatic red teaming pipelines and chain of utterances benchmarks show that iterative exposure to curated adversarial prompts can harden models yet still leave high jailbreak rates in both open and closed systems [10] [2]. Dataset auditing and discrete optimization frameworks such as ARCA demonstrate that multi LLM search can efficiently surface toxic artifacts and "toxic tokens," enabling adversarial training and pre deployment screening [19], and recent work on proxy/guardrail architectures and chain of thought reasoning suggests that dedicated safety wrappers and multi step judgment can improve attack detection without fully resolving robustness utility trade offs [38] [49] [23]. For evaluation, Shen [32] and Chu [9] construct large scale in the wild jailbreak datasets spanning diverse harmful domains, while additional attack families such as direct requests, persuasive adversarial prompts, and AutoDAN/AutoPrompt variants further stress test defenses [46] [51]. Building on this corpus of attack taxonomies, adversarial generation techniques, dataset auditing tools, guard rail architectures, and safety benchmarks [18] [45] [26] [30] [41], the present dissertation aims to synthesize these "ingredients" into a modular, SLM focused framework that integrates multi LLM dataset auditing, adversarial artifact conditioning, and chain of thought guardrail wrappers to achieve LLM grade safety robustness while preserving domain performance and SLM class latency.

III. Proposed Architecture

Large Language Models (LLMs) enable security threats that were previously impractical, including scalable automated extortion and highly convincing fabricated conversations across text, voice and video modalities [4] [43] [34]. Because broad adoption depends on the ability to control harmful content, attacks, and defenses on LLMs are an active area; however, a particularly critical risk arises from fine tuning with harmful or adversarial data, which can degrade or remove safety behaviors.

A recent deployment trend favors Small Language Models (SLMs; $< 7B$ parameters) over very large models to enable on device deployment and lower inference latency [46] [36]. SLMs provide practical benefits of lower compute memory

consumption, faster inference, IoT compatibility, cost efficiency, improved interpretability, and debuggability, especially when fine tuned for domain tasks. The main constraint is that SLM fine tuning requires high quality "clean" datasets, often requiring expensive human review. To reduce this cost, some pipelines (e.g., used in DeepSeek model training) incorporate a supervised LLM teacher/judge who filters, labels, or scores candidate training samples to produce a higher quality fine tuning set [14].

Despite these advantages, SLMs exhibit weaker safety robustness than their larger counterparts due to lower representational capacity and reduced ability to internalize nuanced refusal behaviors learned during alignment (e.g., RLHF or supervised alignment). Empirically, jailbreak research reports that small models (e.g. LLaMA-7B, Pythia-6.9B) can show 30–50% higher attack success rates than larger models ($\geq 13B$) under prompt injection and obfuscation settings [52]. Public evaluations also indicate high jailbreakability: Open LLM Leaderboard style tests report 60% attack success rates against models such as Mistral-7B (base) and TinyLlama-1.1B-Chat, even with simple indirect "role play" jailbreak prompts [46].

Own experiment (retained details): In our internal tests, we evaluated TinyLlama-1.1B-Chat-v1.0 using a DAN jailbreak dataset plus a forbidden questions list, and scored the results with the DAN ChatGLMEval evaluator [32]. In this setup, TinyLlama reached an Attack Success Rate (ASR) of 98%, indicating near total failure of refusal behavior under the tested jailbreak conditions.

A contributing factor is that SLMs are frequently trained without RLHF (often due to resource constraints) and/or exposed to less adversarial data during fine tuning [29] [5] [7] [39]. Consequently, they tend to rely on external safety layers (prompt filters, safety system prompts) rather than internalized safety behaviors; these external defenses are relatively easy to bypass using obfuscation, encoding, or translation [35] [46] [15]. Moreover, even small amounts of harmful fine tuning data can be sufficient to compromise aligned behavior: prior work shows that as few as 340 adversarial examples can significantly remove the protections introduced by RLHF [44]. Current research explores fine tuning strategies aimed at improving harmful content rejection without compromising response speed, which is a key deployment requirement for SLMs [24].

This work proposes a modular, wrapper based safety architecture that mediates interactions between a user and a task specialized fine tuned Small Language Model (SLM) by inserting an additional SLM based guardrail layer. The goal is to detect and prevent unsafe behavior triggered by adversarial prompts (e.g., jailbreaks, prompt injection) or harmful requests, while keeping the task model unchanged ("black box").

A. Architecture

The system comprises four main components:

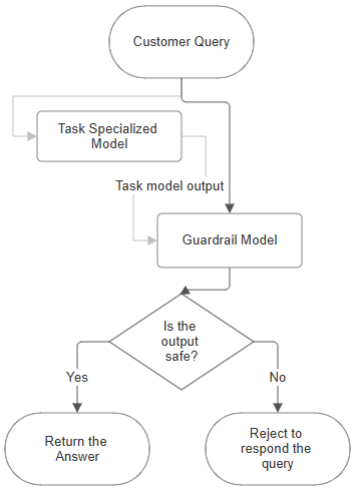

Fig. 1. Guardrails Architecture with Embedded SLM for Safe Model User Interaction

- User Interface Layer: Accepts natural language queries from the end user.
- Guardrail Wrapper (SLM based): A lightweight safety layer that performs two stage gating:
  - Input analysis: screens user queries before they reach the task model.
  - Output evaluation: inspects the generated response before it is returned to the user.
- Task Specialized Fine tuned SLM (Black box): The primary generator, not modified by the safety mechanism to preserve original and fine tuned capabilities.
- Labeling and Response Control Module: Enforces policy decisions based on the guardrail's labels (e.g., allow, block, rewrite/redact, log).

B. Operational Workflow

- Input interception and classification: The guardrail SLM evaluates whether an input is benign or malicious/adversarial (e.g., jailbreak/prompt injection/ethical violations). It also checks for "toxic tokens" identified via prior AutoDAN based analysis of the task model. Flagged inputs can be rejected or routed for user reformulation.
- Task model invocation: Only inputs that pass the first stage are forwarded to the task specialized fine tuned SLM, minimizing interference with its generative behavior.
- Output analysis and labeling: The guardrail evaluates the generated response in context of the original prompt and labels it as:
  - Safe (aligned with responsible use), or
  - Unsafe (complies with malicious intent or reveals harmful/sensitive content).
- Response decision: Safe responses are returned. Unsafe responses are blocked, optionally redacted/rewritten, and may be logged for audit.

C. Design Rationale

- SLM based guardrails are used to reduce latency and computational overhead, enabling real time filtering in constrained or production settings.
- Non intrusive deployment preserves compatibility with closed source/commercial models by leaving the task model untouched.
- Layered (input + output) evaluation mitigates both prompt injection and harmful output leakage in a single integrated workflow.

Application Context (Cybersecurity and High Assurance Domains) The architecture is framed as an LLM hardening mechanism suitable for high assurance environments (e.g., healthcare, finance, legal), supporting governance objectives such as monitoring, auditing, and alignment with responsible AI practices.

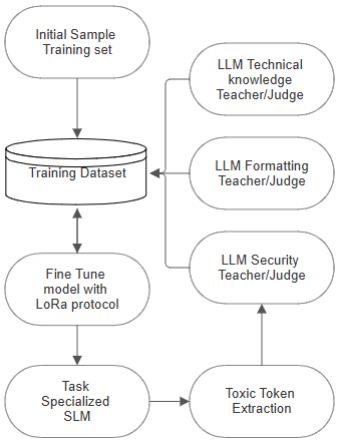

Fig. 2. Task Specialized SLM Fine Tune and Toxic Tokens Extractions

D. Multilayered defense architecture for safe SLM fine tuning

This work proposes a three phases, defense in depth pipeline to enable safe domain adaptation of Small Language Models (SLMs) while improving robustness against prompt based attacks, including jailbreaks, prompt injection, and malicious user queries. The key idea is to (i) harden the training data via multi model scrutiny, (ii) fine tune with safety oriented objectives, and (iii) empirically stress test the resulting SLM using an automated jailbreak method to discover and operationalize newly emerging attack triggers.

1) Phase 1: Dataset creation with a multi LLM audit pipeline: Prior to fine tuning, the training corpus is curated through a multi agent auditing workflow designed to minimize the inclusion of harmful or policy violating content that could later be amplified during instruction tuning. First, raw domain data (e.g., healthcare, legal, customer support) is collected or modified to match target use cases and is pre filtered to remove known vulnerabilities (e.g., toxic language, unsafe procedural guidance, dangerous code patterns). Second, each candidate training sample is evaluated independently by multiple distinct LLM auditors (e.g., GPT-4, Claude, Gemini), leveraging diverse model behaviors to reduce single model blind spots. Third, the final training set is synthesized using an intersection rule: only samples approved by all auditors are retained. Optionally, a meta LLM judge performs a final reasoning based verification pass to improve alignment and robustness.

2) Phase 2: Secure fine tuning of the SLM: The audited dataset is then used to fine tune a target SLM (e.g., TinyLLaMA, Mistral, or Phi family models) with objectives aimed at maintaining instruction following utility while reducing harmful compliance. The fine tuning strategy emphasizes (i) instruction tuning with safe prompt–response pairs, (ii) contrastive learning signals that separate harmful from safe behaviors, and (iii) optional reinforcement learning from AI feedback (RLAIF) to further shape refusal and safe completion policies when human feedback is unavailable or limited.

3) Phase 3: Post finetuning jailbreak evaluation and toxicity token extraction: After fine-tuning, the model is evaluated using AutoDAN, an automated discrete optimization jailbreak framework that searches for adversarial prompts capable of eliciting unsafe responses [51]. Successful adversarial triggers are then used for toxicity token / malicious pattern extraction, producing a set of high risk tokens or prompt fragments associated with safety failures. These triggers are (i) logged into a Toxic Token Repository, (ii) used to train a guardrail or adversarial detector, and (iii) optionally deployed at inference time through token filtering and/or response rewriting to mitigate recurring jailbreak patterns.

IV. Experiment Overview: Securing Fine Tuned SLM for JavaScript QA Tasks

This proof of concept evaluates whether a lightweight, SLM based guardrail wrapper can increase safety robustness (jailbreak/toxicity resistance) while maintaining task quality and preserving the latency advantages of small language models (SLMs) relative to larger LLMs. The target domain is JavaScript library question answering (QnA) [55].

The system uses a two-stage inference wrapper around a fine-tuned domain SLM:

- Input screening (pre generation): A dedicated guardrail/alignment SLM evaluates the user prompt for policy violating intent (including DAN style jailbreak intent) and for the presence of adversarial "toxic trigger tokens" discovered during red-teaming. Prompts failing screening can be blocked, rewritten, or routed to a safe response pathway (implementation dependent).
- Task generation: The screened prompt is answered by the domain task SLM fine tuned for JavaScript library QnA.
- Output vetting (post generation): The guardrail SLM reevaluates the generated answer to detect disallowed content (e.g., policy violations, unsafe code guidance, or compliance with jailbreak instructions). If unsafe, the wrapper can refuse, sanitize, or trigger regeneration.

A. Models

- Task model (fine-tuned domain SLM): `Edcastro/tinyllama-edcastr_JavaScript-v2` [53]
- Guardrail model (alignment/safety SLM): `Edcastro/tinyllama-edcastr_Guardrail-v1` [54]

This design keeps safety checks model-local and lightweight, avoiding the cost of calling a larger external LLM for moderation, and enabling low-latency deployment in domain QnA settings.

B. Data assets and alignment signal construction

Three main assets are used:

- JavaScript Library QnA dataset (training/evaluation) [55] for domain capability and task quality.
- Guardrail tuning dataset [56] curated specifically for alignment/safety behavior.
- Wrapper experimentation notebook [57] used to run the two stage pipeline and evaluate end to end behavior.

The guardrail tuning dataset [56] is derived from two key sources of safety signal:

- LLM as Teacher/judge ("GPT-judges") labeling to score/identify unsafe or policy noncompliant behavior.
- AutoDAN style adversarial analysis, used to discover toxic trigger tokens and adversarial patterns that emerge post fine tuning or in out of the box models. These discovered tokens/patterns are then incorporated into guardrail training and/or wrapper heuristics, improving robustness to known jailbreak strings.

C. Experimental environment

Experiments are run on Azure ML using `Standard_NC64as_T4_v3`:

- 4 × NVIDIA Tesla T4 GPUs
- 64 CPU cores
- 440 GB RAM

This environment supports measuring whether the wrapper's additional inference steps (input screening + output vetting) still preserve the expected latency advantage of SLMs.

D. Adversarial testing and evaluation metrics

Robustness is assessed using two complementary adversarial methods:

- AutoDAN style jailbreak probing: Used primarily for discovery identifying toxic trigger tokens and jailbreak templates that can cause policy violations.
- Attack Success Rate (ASR): The principal robustness metric, measured on a blended forbidden-question set plus DAN style prompts. ASR captures the proportion of adversarial prompts that successfully elicit disallowed behavior.

In addition to safety/ASR, the PoC evaluates:

- Task quality on the JavaScript library QnA evaluation (e.g., correctness, completeness, and relevance; judged either automatically, via LLM as judge, or via human evaluation depending on the thesis protocol).
- Latency impact of the two stage wrapper relative to running the task SLM alone, and relative to larger LLM baselines (if included in the thesis).

If you paste the thesis' numeric results tables/paragraphs (ASR before/after wrapper, task scores, latency), I can rewrite them in paper ready form while keeping all reported values intact.

V. Results

TABLE I
TASK MODEL ALONE (TINYLLAMA JAVASCRIPT QNA SLM)

	True Positive	***True Negative***
Predicted Positive	1090	94
Predicted Negative	481	3

$$TP = 1090, FP = 94, FN = 481, TN = 3$$

$$(Total = 1668; Positives = 1571; Negatives = 97)$$

Metrics:
Precision $= 0.9206$, Recall (Positive) $= 0.6938$, Accuracy $= 0.6553$, F1 $= 0.7913$

Derived Safety-Relevant Metrics:
Negative Recall (True Negative Rate) $= TN/(TN + FP) = 3/97 = 0.0309$, False Positive Rate on unsafe inputs $= 0.9691$, Balanced Accuracy $= (Recall_pos + Recall_neg)/2 = (0.6938 + 0.0309)/2 = 0.3624$, Matthews Correlation Coefficient (MCC) ≈ -0.142 (weak / inverted due to near-total failure on unsafe class)

Interpretation/Insights:
The fine-tuned JavaScript SLM delivers decent precision on benign QnA but is extremely poor at correctly rejecting unsafe inputs failing on 97% of them. This mirrors the known pattern where small domain-tuned models over-generalize helpfulness and lose internalized refusal behavior after fine-tuning.

Security Robustness (AutoDAN ASR):
Vanilla TinyLlama base ASR: 98% and the Fine-tuned JavaScript SLM ASR: 63% (Absolute reduction: 35 percentage points and Relative reduction vs base: 36% lower ASR) Interpretation/Insights: Even without a guardrail wrapper, the domain fine-tune reduced but did not eliminate susceptibility to automated jailbreak prompts. Many residual failure modes likely reflect incomplete coverage of harmful patterns in the training data and catastrophic forgetting of safety signals.

TABLE II
GUARDRAIL WRAPPER (TASK SLM AND GUARDRAIL SLM)

	True Positive	*True Negative*
Predicted Positive	1220	22
Predicted Negative	351	75

$$TP = 1220, FP = 22, FN = 351, TN = 75 (Total = 1668)$$

Metrics:
Precision = 0.9823, Recall (Positive) = 0.7766, Accuracy = 0.7764, F1 = 0.8674

Derived Safety-Relevant Metrics:
Negative Recall (True Negative Rate) $= 75/97 = 0.7732$, False Positive Rate on Unsafe Inputs $= 0.2268$, Balanced Accuracy $= (0.7766 + 0.7732)/2 = 0.7749$, MCC ≈ 0.295 (improves from negative baseline; now meaningfully correlated with true labels)

Comparative Deltas vs Task Model Alone:
$+6.3\%$ absolute Recall gain (positive class), $+12.1\%$ absolute Accuracy gain, $+7.6\%$ absolute F1 gain, Negative Recall improved by $+74.2$ percentage points $(0.03 \rightarrow 0.77)$, False Positive Rate on unsafe inputs fell by 74 percentage points, Balanced Accuracy more than doubled $(0.36 \rightarrow 0.77)$, Precision improved from $0.92 \rightarrow 0.98$ despite Recall gain (rare and valuable)

Interpretation/Insights:
The wrapper dramatically increases the system's ability to correctly block unsafe / out-of-scope content while also improving allowed benign utility. In other words, we reduced over-compliance AND under-responsiveness simultaneously. This speaks to synergistic effects when an alignment-oriented SLM screens and post-validates a domain SLM's output.

Latency:
The average pipeline latency of our PoC (Task SLM and Guardrail SLM wrapper) is 0.614 seconds, while the same dataset running in the same machine with a bigger model Llama-3.1-8B single-model is 10.740 seconds, which Speedup: 17.5 faster end-to-end response than Llama-3.1-8B. Interpretation/Insights: The dual-SLM wrapper retains the small-model responsiveness advantage even with two passes (input and output). This validates the premise that layered safety does not have to sacrifice interactivity when implemented with SLMs.

VI. CONCLUSION

This paper investigated the security gap that emerges when Small Language Models (SLMs) are fine tuned for high-utility domain behavior: safety priors weaken, refusal reliability collapses, and the resulting models become disproportionately susceptible to prompt-based jailbreaks and fine tuning–driven safety erosion. To address this, we proposed a modular security alignment framework designed for resource-bounded deployment, combining (i) multi-model dataset auditing, (ii) post-finetuning adversarial artifact discovery and toxic-token conditioning, and (iii) a lightweight SLM-based guardrail wrapper that performs both pre-generation intent screening and post-generation output vetting around an otherwise unchanged task SLM.

Empirically, the JavaScript QnA proof of concept demonstrates that wrapper based mediation can substantially improve safety relevant behavior without sacrificing the latency advantages that motivate SLM adoption. Relative to the task SLM alone, the guardrail wrapper improved overall classification utility (Precision $0.92 \rightarrow 0.98$, F1 $0.79 \rightarrow 0.87$) while dramatically increasing correct handling of unsafe inputs (True Negative Rate $0.03 \rightarrow 0.77$; Balanced Accuracy $0.36 \rightarrow 0.77$), converting an effectively nonfunctional unsafe-class rejection regime into a meaningfully reliable decision boundary. In parallel, the end to end wrapper latency remained sub-second (0.614 s) and provided a $\sim 17.5\times$ speedup compared to a larger single model baseline (Llama-3.1-8B at 10.740 s) on the same hardware, supporting the central claim that defense in depth can be feasible under strict responsiveness constraints.

These results suggest that safety for fine tuned SLMs is achievable through modular composition rather than monolithic realignment, enabling organizations to preserve domain performance and deployment efficiency while materially reducing unsafe compliance pathways. Future work should expand evaluation to broader harm taxonomies and adaptive attackers, report wrapper level jailbreak Attack Success Rate (ASR) under automated red teaming (e.g., AutoDAN and transfer attacks), and study robustness under distribution shift, multilingual obfuscation, and adversarial fine tuning of the guardrail itself. Additional directions include calibrated uncertainty and abstention, policy grounded rewrite strategies (beyond blocking), and tighter coupling between toxic token repositories and continual adversarial training to improve long term alignment persistence.

REFERENCES

[1] Andriushchenko, M., Croce, F., Flammarion, N. (2024). Jailbreaking leading safety-aligned llms with simple adaptive attacks. arXiv preprint arXiv:2404.02151.

[2] Bhardwaj, R., Poria, S. (2023). Red-teaming large language models using chain of utterances for safety-alignment. arXiv preprint arXiv:2308.09662.

[3] Bianchi, F., Suzgun, M., Attanasio, G., Röttger, P., Jurafsky, D., Hashimoto, T., and Zou, J. (2023). Safety-tuned llamas: Lessons from improving the safety of large language models that follow instructions. arXiv preprint arXiv:2309.07875.

[4] Brundage, M., Avin, S., Clark, J., Toner, H., Eckersley, P., Garfinkel, B., ... and Amodei, D. (2018). The malicious use of artificial intelligence: Forecasting, prevention, and mitigation. arXiv preprint arXiv:1802.07228.

[5] Casper, S., Davies, X., Shi, C., Gilbert, T. K., Scheurer, J., Rando, J., ... and Hadfield-Menell, D. (2023). Open problems and fundamental limitations of reinforcement learning from human feedback. arXiv preprint arXiv:2307.15217.

[6] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., and Wong, E. (2024). Jailbreaking black box large language models in twenty queries. https://arxiv. org/abs/2310.08419

[7] Chaudhari, S., Aggarwal, P., Murahari, V., Rajpurohit, T., Kalyan, A., Narasimhan, K., ... and Castro da Silva, B. (2024). Rlhf deciphered: A critical analysis of reinforcement learning from human feedback for llms. ACM Computing Surveys.

[8] Chennabasappa, S., Nikolaidis, C., Song, D., Molnar, D., Ding, S., Wan, S., ... and Saxe, J. (2025). Llamafirewall: An open source guardrail system for building secure ai agents. arXiv preprint arXiv:2505.03574.

[9] Chu, J., Liu, Y., Yang, Z., Shen, X., Backes, M., and Zhang, Y. (2024). Comprehensive assessment of jailbreak attacks against llms. https://arxiv.org/abs/2402.05668

[10] Deng, B., Wang, W., Feng, F., Deng, Y., Wang, Q., and He, X. (2023). Attack prompt generation for red teaming and defending large language models. arXiv preprint arXiv:2310.12505.

[11] Du, Y., Zhao, S., Ma, M., Chen, Y., and Qin, B. (2023). Analyzing the inherent response tendency of llms: Real-world instructions-driven jailbreak. arXiv preprint arXiv:2312.04127.

[12] Geisler, S., Wollschläger, T., Abdalla, M. H. I., Gasteiger, J., and Günnemann, S. (2024). Attacking large language models with projected gradient descent. arXiv preprint arXiv:2402.09154.

[13] Guo, X., Yu, F., Zhang, H., Qin, L., and Hu, B. (2024). Cold-attack: Jailbreaking llms with stealthiness and controllability. arXiv preprint arXiv:2402.08679.

[14] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., ... and He, Y. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948.

[15] Gupta, A. (2025). Prompt Injection Generation Using Small Language Models with Reinforcement Learning with Artificial Intelligence Feedback (Doctoral dissertation, Massachusetts Institute of Technology).

[16] Hayase, J., Borevković, E., Carlini, N., Tramèr, F., and Nasr, M. (2024). Query-based adversarial prompt generation. Advances in Neural Information Processing Systems, 37, 128260-128279.

[17] Huang, Y., Gupta, S., Xia, M., Li, K., and Chen, D. (2023). Catastrophic jailbreak of open-source llms via exploiting generation. arXiv preprint arXiv:2310.06987.

[18] Islam, M. R., Dhar, N., Deng, B., Nguyen, T. N., He, S., and Suo, K. (2024, November). Characterizing and Understanding the Performance of Small Language Models on Edge Devices. In 2024 IEEE International Performance, Computing, and Communications Conference (IPCCC) (pp. 1-10). IEEE.

[19] Jones, E., Dragan, A., Raghunathan, A., and Steinhardt, J. (2023, July). Automatically auditing large language models via discrete optimization. In International Conference on Machine Learning (pp. 15307-15329). PMLR.

[20] Lermen, S., Rogers-Smith, C., and Ladish, J. (2023). LoRA fine-tuning efficiently undoes safety training in Llama 2-Chat 70B. arXiv.

[21] Li, S., Ngai, E. C. H., Ye, F., and Voigt, T. (2024). Peft-as-an-attack! jailbreaking language models during federated parameter-efficient fine-tuning. arXiv preprint arXiv:2411.19335.

[22] Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., and Xiao, C. (2024). Autodan-turbo: A lifelong agent for strategy self-exploration to jailbreak llms. https://arxiv.org/abs/2410.05295

[23] Mangaokar, N., Hooda, A., Choi, J., Chandrashekaran, S., Fawaz, K., Jha, S., and Prakash, A. (2024). Prp: Propagating universal perturbations to attack large language model guard-rails. arXiv preprint arXiv:2402.15911.

[24] Matan, P., and Velvizhy, P. (2025, April). A Comprehensive Review of Supervised Fine-Tuning for Large Language Models in Creative Applications and Content Moderation. In 2025 International Conference on Inventive Computation Technologies (ICICT) (pp. 1294-1299). IEEE.

[25] Mehrotra, A., Zampetakis, M., Kassianik, P., Nelson, B., Anderson, H., Singer, Y., and Karbasi, A. (2024). Tree of attacks: Jailbreaking black-box llms automatically. https://arxiv.org/abs/2312.02119

[26] Mekala, D., Nguyen, A., and Shang, J. (2024). Smaller language models are capable of selecting instruction-tuning training data for larger language models. arXiv preprint arXiv:2402.10430.

[27] Mylrea, M. (2025). The generative AI weapon of mass destruction: Evolving disinformation threats, vulnerabilities, and mitigation frameworks. In Interdependent Human-Machine Teams (pp. 315-347). Academic Press.

[28] Naidu, G., Zuva, T., and Sibanda, E. M. (2023, April). A review of evaluation metrics in machine learning algorithms. In Computer science on-line conference (pp. 15-25). Cham: Springer International Publishing.

[29] Parthasarathy, V. B., Zafar, A., Khan, A., and Shahid, A. (2024). The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities. arXiv preprint arXiv:2408.13296.

[30] Peng, B., Chen, K., Niu, Q., Bi, Z., Liu, M., Feng, P., ... and Yin, C. H. (2024). Jailbreaking and mitigation of vulnerabilities in large language models. arXiv preprint arXiv:2410.15236.

[31] Qi, X., Zeng, Y., Xie, T., Chen, P. Y., Jia, R., Mittal, P., and Henderson, P. (2023). Fine-tuning aligned language models compromises safety, even when users do not intend to!. arXiv preprint arXiv:2310.03693.

[32] Shen, X., Chen, Z., Backes, M., Shen, Y., and Zhang, Y. (2024, December). " do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (pp. 1671-1685).

[33] Sitawarin, C., Mu, N., Wagner, D., and Araujo, A. (2024). Pal: Proxy-guided black-box attack on large language models. arXiv preprint arXiv:2402.09674.

[34] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., and Ortega-Garcia, J. D. beyond: A survey of face manipulation and fake detection. arXiv 2020. arXiv preprint arXiv:2001.00179.

[35] Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., ... and Wang, S. (2024). A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. arXiv preprint arXiv:2411.03350.

[36] Wang, H., Li, H., Huang, M., and Sha, L. (2024). Asetf: A novel method for jailbreak attack on llms through translate suffix embeddings. arXiv preprint arXiv:2402.16006.

[37] Wang, J., Liu, Z., Park, K. H., Jiang, Z., Zheng, Z., Wu, Z., ... and Xiao, C. (2023). Adversarial demonstration attacks on large language models. arXiv preprint arXiv:2305.14950.

[38] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., ... and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems, 35, 24824-24837.

[39] Whitmore, S., Harrington, C., and Pritchard, E. (2024). Assessing the ineffectiveness of synthetic reinforcement learning feedback in fine-tuning large language models.

[40] Yang, X., Wang, X., Zhang, Q., Petzold, L., Wang, W. Y., Zhao, X., and Lin, D. (2023). Shadow alignment: The ease of subverting safely-aligned language models. arXiv preprint arXiv:2310.02949.

[41] Yi, S., Liu, Y., Sun, Z., Cong, T., He, X., Song, J., ... and Li, Q. (2024). Jailbreak attacks and defenses against large language models: A survey. arXiv preprint arXiv:2407.04295.

[42] Yu, J., Lin, X., Yu, Z., and Xing, X. (2024). Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts. https://arxiv.org/abs/2309.10253

[43] Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., and Choi, Y. (2019). Defending against neural fake news. Advances in neural information processing systems, 32.

[44] Zhan, Q., Fang, R., Bindu, R., Gupta, A., Hashimoto, T., and Kang, D. (2023). Removing rlhf protections in gpt-4 via fine-tuning. arXiv preprint arXiv:2311.05553.

[45] Zhang, Q., Liu, Z., and Pan, S. (2025). The rise of small language models. IEEE Intelligent Systems, 40(1), 30-37.

[46] Zhang, W., Xu, H., Wang, Z., He, Z., Zhu, Z., and Ren, K. (2025). Can Small Language Models Reliably Resist Jailbreak Attacks? A Comprehensive Evaluation. arXiv preprint arXiv:2503.06519.

[47] Zhang, Z., Shen, G., Tao, G., Cheng, S., and Zhang, X. (2023). Make them spill the beans! coercive knowledge extraction from (production) llms. arXiv preprint arXiv:2312.04782.

[48] Zhao, X., Yang, X., Pang, T., Du, C., Li, L., Wang, Y. X., and Wang, W. Y. (2024). Weak-to-strong jailbreaking on large language models. arXiv preprint arXiv:2401.17256.

[49] Zheng, A., Rana, M., and Stolcke, A. (2024). Lightweight safety guardrails using fine-tuned bert embeddings. arXiv preprint arXiv:2411.14398.

[50] Zhou, Y., Huang, Z., Lu, F., Qin, Z., and Wang, W. (2024). Don't Say No: Jailbreaking LLM by Suppressing Refusal. arXiv preprint arXiv:2404.16369.

[51] Zhu, S., Zhang, R., An, B., Wu, G., Barrow, J., Wang, Z., ... and Sun, T. (2023). AutoDAN: interpretable gradient-based adversarial attacks on large language models. arXiv preprint arXiv:2310.15140.

[52] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043.
[53] https://huggingface.co/Edcastro/tinyllama-edcastr_JavaScript-v2 (accessed Jan. 5, 2026).
[54] https://huggingface.co/Edcastro/tinyllama-edcastr_Guardrail-v1 (accessed Jan. 5, 2026).
[55] https://github.com/Seb4stian/NSU_SLM_SecureFineTune/blob/main/JavaScriptModelTraining/JavaScriptLibraryDataset_with_responses.jsonl (accessed Jan. 5, 2026).
[56] https://github.com/Seb4stian/NSU_SLM_SecureFineTune/blob/main/Guardrail%20SLM%20Training%20Dataset%20preparation/Guardrail_dataset.jsonl (accessed Jan. 5, 2026).
[57] https://github.com/Seb4stian/NSU_SLM_SecureFineTune/blob/main/WrapperModel/Guardrail_training.ipynb (accessed Jan. 5, 2026).
[58] https://huggingface.co/meta-llama/Llama-3.1-8B (accessed Jan. 5, 2026).